

UVM Sequence Item, Reporting Macros and Phases

Yasir Farooq

UVM Transaction

- A class object interpreted as packet of data
- Simple example can be a read/write transaction on a bus
- UVM class library has `uvm_transaction` base class

```
class trans extends uvm_transaction
    rand bit [31:0] data; // Data
    rand bit [4:0] addr;  // Address
    ....
    constraint c {addr >= 5'h0 && addr <= 5'h10; }; //
constraints
endclass
```

Sequence Items

- UVM provides base class `uvm_sequence_item`
- It contains the data fields required to be a part of the transaction packet
- It may have constraints on the fields or randomization in fields
- `uvm_object` methods are used on this class objects

Sequence Item – Methods

Some of the commonly used factory methods for sequence items are:

- **create():** The create method allocates a new object of the same type as this object and returns it via *uvm_object* handle

seq_item = seq_item_classname::type_id::create("seq_item")

- **print():** This method deep-prints this object's properties in a format and manner governed by govern printer argument

seq_item.print()

Sequence Item – Methods (cont'd)

- **copy():** Makes this object a copy of a specified object
seq_item1.copy(seq_item2)
- **clone():** Creates and returns an exact copy of this object. clone = create() + copy()
\$cast(seq_item1, seq_item2.clone())
- **compare():** Deep compares members of a data object. Returns 1 on a match, 0 otherwise
seq_item1.compare(seq_item2)

Example

```
class yapp_packet extends uvm_sequence_item;  
    rand bit [5:0] length;  
    ... // other field declarations
```

```
function new (string name = "yapp_packet");  
    super.new(name);  
endfunction
```

```
`uvm_object_utils_begin(yapp_packet)  
    `uvm_field_int( length, UVM_ALL_ON)  
    // more field automation  
`uvm_object_utils_end
```

```
endclass : yapp_packet
```

Inherit from
uvm_sequence_item

Data constructor with string
name argument and default value

Must call super.new(name);

Must have utility macro block

Argument is name of class

Enable automation for
yapp_packet fields

Example

Every data item property should have a field automation macro

```
typedef enum bit {GOOD_PARITY, BAD_PARITY} parity_t;
class yapp_packet extends uvm_sequence_item;
...
rand bit [7:0] payload[];
rand bit [7:0] parity;
rand parity_t parity_type;
rand int packet_delay;

// field declaration automation flags
`uvm_object_utils_begin(yapp_packet)
...
`uvm_field_array_int( payload, UVM_ALL_ON )
`uvm_field_int( parity, UVM_ALL_ON + UVM_BIN )
`uvm_field_enum( parity_t, parity_type, UVM_ALL_ON )
`uvm_field_int( packet_delay, UVM_ALL_ON + UVM_NOCOMPARE )
`uvm_object_utils_end

endclass: yapp_packet
```

Enables automation
for yapp_packet fields

Print radix

UVM_NOCOMPARE overrides
UVM_COMPARE set by UVM_ALL_ON

Print Automation Method

```
yapp_packet p1;  
int ok;  
string str;  
  
initial begin  
    p1 = new("p1");  
    ok = p1.randomize();  
  
    // default uvm_default_table_printer  
    p1.print();  
    // change print policy  
    p1.print(uvm_default_tree_printer);  
  
    str = p1.sprint();  
    ...  
end
```

Creates string but doesn't print

Tree view

- Customized print views can also be defined

Table view: (default)

Name	Type	Size	Value
p1	yapp_packet	-	@1150
length	integral	6	'd14
addr	integral	2	' d2
parity	integral	8	'b110001
payload	da(integral)	5	-
[0]	integral	8	'h2d
[1]	integral	8	'h95
...			

```
p1: (yapp_packet@1150) {  
    length: 'd14  
    addr: 'd2  
    parity: 'b110001  
    payload: {  
        [0]: 'h2d  
        [1]: 'h95  
        ...  
    }  
}
```

Automation flags
determine print radix

UVM Messaging and Reporting Overview

- Messaging Components:
 - Severity: Indicates importance (Fatal, Error, Warning, Info).
 - Verbosity: Controls message filtering (None, Low, Medium, High, Full, Debug).
 - Simulation Handling Behavior: Defines simulator actions (Exit, Count, Display, Log, No Action).
- Purpose: Messaging Components:
 - Enables structured reporting of testbench messages.
 - Provides debugging and information control via macros.

UVM Reporting Macros

- Macros for Message Reporting:
 - `uvm_info(ID, MSG, VERBOSITY)` → Informative messages.
 - `uvm_warning(ID, MSG)` → Warnings.
 - `uvm_error(ID, MSG)` → Errors that do not halt simulation.
 - `uvm_fatal(ID, MSG)` → Critical errors that stop simulation.
- Verbosity Control:
 - Messages are printed based on verbosity settings.
 - `+UVM_VERBOSITY=UVM_<LEVEL>` .
 - `uvm_fatal`, `uvm_error`, and `uvm_warning` are not affected by verbosity settings.

UVM Verbosity Levels (Lowest to Highest)

- **UVM_NONE**
 - All messages are printed.
 - Cannot be overridden.
- **UVM_LOW**
 - Prints only essential messages.
 - Typically used for minimal logging in production runs.
- **UVM_MEDIUM** (Default)
 - Provides balanced logging.
 - Includes key messages for debugging and status updates.
- **UVM_HIGH**
 - Prints detailed debugging information. Useful during testbench development.
- **UVM_DEBUG**
 - Most detailed level, used for deep debugging. Includes low-level internal UVM execution logs.

UVM Reporting Macros Examples

```
`uvm_info("PACKET_MON", "Packet received successfully", UVM_LOW)
```

```
`uvm_info("TRANSACTION", $sformatf("Transaction ID: %0d, Data:  
%0h", trans_id, data), UVM_MEDIUM)
```

```
`uvm_warning("CONFIG", "Configuration value out of range, using  
default settings")
```

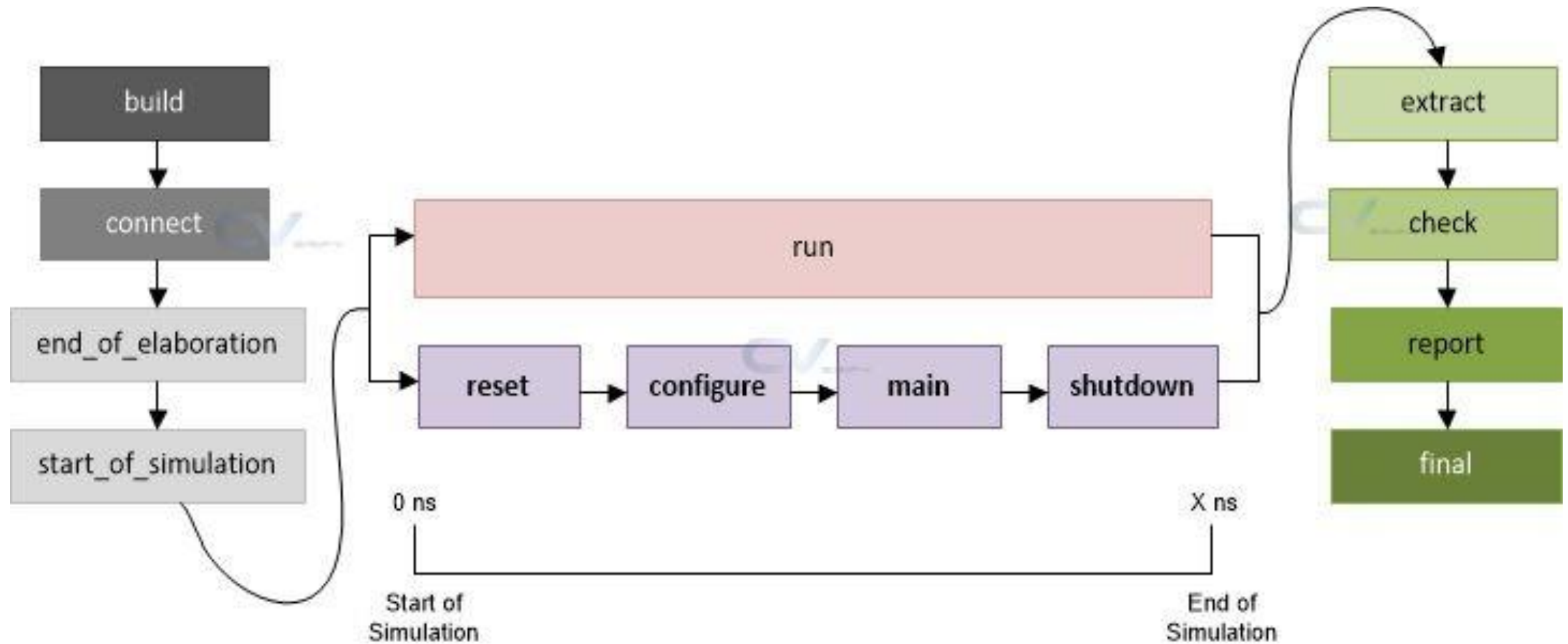
```
`uvm_error("BUS_MON", "Unexpected bus transaction detected!")
```

```
`uvm_fatal("MEMORY", "Memory corruption detected! Simulation will stop.")
```

UVM Phases

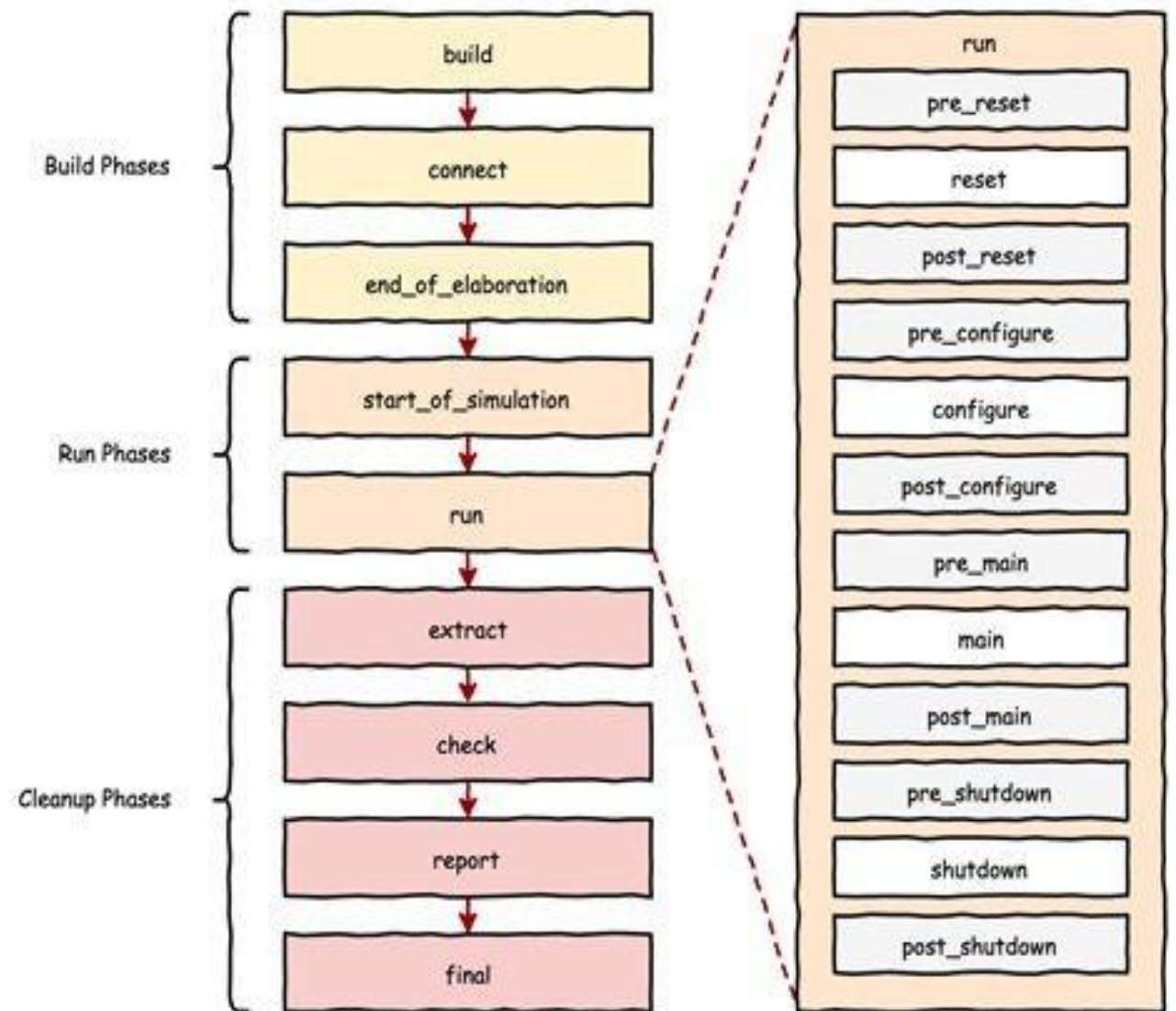
- Synchronising Mechanism:
 - UVM phases are call backs, which are called in a particular fashion
 - It keeps the execution in sequence
- Three types of UVM phases are,
 - Build Phases
 - Run Phases
 - Cleanup Phases

UVM Phases



UVM Phases

- UVM phases are virtual functions or virtual tasks defined in the base classes
 - All are functions except `run_phase()`



Lab Exercise

Lab 2 Understanding `uvm_sequence_item`

To Learn how to create and use `uvm_sequence_item` with `uvm_field_*` macros in UVM